

Lazy Loading

Ship less JavaScript on first load — a route's code loads only when it's actually visited, instead of shipping every page's component inside the main JavaScript bundle from the very first load. This guide also covers Guards and Resolvers, the two techniques you'll pair it with on real routes.

1 · THE PROBLEM

Every route ships eagerly, whether it's visited or not

With a normal eager `import`, Angular bundles a route's component straight into the main bundle — the file the browser must download and parse before the app can even start, even for pages the user may never open.

2 · ONE-LINE CHANGE, REAL PAYOFF

Eager imports vs. `loadComponent`

✗ Before — every component is in the main bundle

```
import { Products } from './exercises/product-card/product-card';
import { Checkout } from './exercises/checkout/checkout';

export const routes: Routes = [
  { path: 'products', component: Products },
  { path: 'checkout', component: Checkout },
];
```

✓ After — code loads only when the route is visited

```
export const routes: Routes = [
  { path: 'products', loadComponent: () =>
    import('./exercises/product-card/product-card')
      .then(m => m.Products) },
  { path: 'checkout', loadComponent: () =>
    import('./exercises/checkout/checkout')
      .then(m => m.Checkout) },
];
```

3 · WHY IT MATTERS

Smaller initial bundle, faster first paint

1

Smaller main bundle

The browser downloads and parses less JavaScript before the app becomes interactive.

2

Loaded on demand

A route's chunk is fetched only the moment the user actually navigates to it.

3

One-line change

`component: X` becomes `loadComponent: () => import(...)` — no restructuring needed.

CanActivate — controlling when a route may activate

Guards and Resolvers control what happens right as a lazily-loaded (or any) route activates — should this navigation be allowed, and what data does it need first?

```
export const cartNotEmptyGuard: CanActivateFn = () => {
  const cartService = inject(CartService);
  const router = inject(Router);

  if (cartService.cartItems().length > 0) {
    return true;
  }
  return router.parseUrl('/');
};

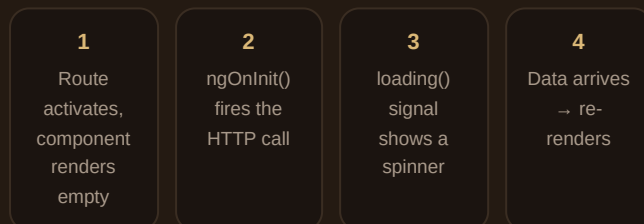
// app.routes.ts
{ path: 'checkout', canActivate: [cartNotEmptyGuard], ... }
```

A guard returns `true` (allow), `false` (block), or a `UrlTree` (redirect instead). Returning a `UrlTree` avoids a flash of the blocked page before redirecting. Functional guards use `inject()` — no class, no constructor, easy to unit test.

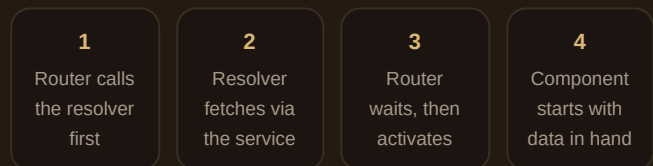
5 · BONUS · RESOLVERS

Fetch first, render once

✗ Without a resolver



✓ With a resolver



```
export const productResolver: ResolveFn<Product> = (route) =>
  inject(ProductsService).getProductById(
    Number(route.paramMap.get('id'))
  );

// app.routes.ts
{ path: 'products/:id', component: ProductDetails,
  resolve: { product: productResolver } }
```

No manual loading state needed — the component starts with data already in hand, because the Router waited for the resolver before activating it.